Оценка требований ТЗ

Краткая экспертиза ТЗ

ТЗ описывает классическую **helpdesk / ticketing-систему** с ролевой моделью (Клиент → Оператор → Модератор → Администратор), двусторонней перепиской в заявках, автоназначением оператора, email-уведомлениями, журналом аудита и экспортом в CSV.

Сильные стороны ТЗ:

- Чётко выделены роли и матрица прав (хорошо ложится на RBAC).
- Есть явные нефункциональные требования (цвета статусов, мобильная адаптация, ≤2 с на запросы, подтверждение перед мутациями, хеширование паролей, кросс-браузерность).
- Перечислены сущности и сценарии — можно строить модель данных.
- Критерии приёмки формально проверяемы.

Слабые места / пробелы, которые стоит уточнить:

1. **«Автоматически прикрепляется доступный оператор»** — не определён алгоритм выбора (round-robin? наименее загруженный? по категории?). Это ключевая бизнес-логика, влияющая на архитектуру (потребуется стратегия + конкурентный доступ).
2. **Email-уведомления** — синхронно или асинхронно? Требование «≤2 с на создание» конфликтует с синхронной отправкой почты → напрашивается очередь/асинхронность.
3. **Журнал действий** — что именно логируется (только заявки или все сущности)? «Изменение заявки» как сущность vs. журнал — нужно разграничить audit log и историю изменений.
4. **Поиск** — полнотекстовый по теме/телу? Это влияет на выбор БД (нужен ли FTS/индексы).
5. **Регистрация с отправкой логина-пароля на email** — потенциальная дыра в безопасности (пароль в открытом виде в письме). Лучше — ссылка активации/сброса.
6. **«Незаблокированные клиенты»** — блокировка пользователя упомянута вскользь, но нет функции блокировки в списке «Функции».
7. **Удаление сущностей** — в нефункциональных требованиях упоминается подтверждение при удалении, но в «Функциях» удаление явно не описано (кроме пользователей/категорий/статусов — «изменение», не удаление). Несогласованность.
8. **Масштаб не задан** — нет ожидаемого числа пользователей/заявок, что влияет на выбор архитектуры.

Вывод: ТЗ пригодно для учебного проекта, но требует уточнений по алгоритму назначения оператора, асинхронности уведомлений и семантике журнала.

Вариант 1 (ваш): Слоистая архитектура (Layered / N-tier)

`presentation/controller → service → repository/data access → database`

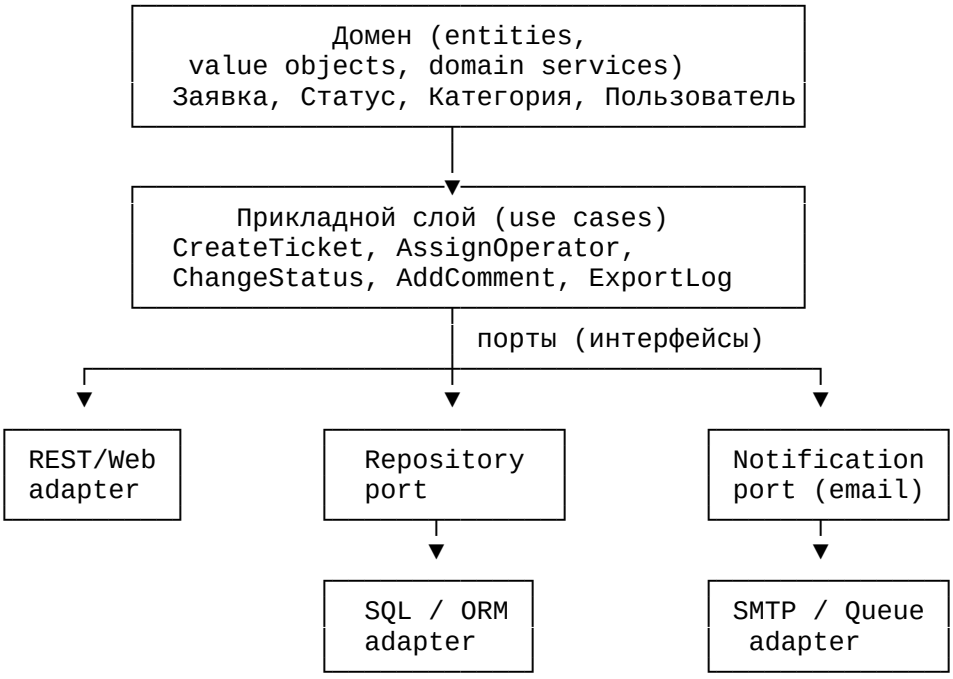
Классическая монолитная слоистая схема. Подходит, когда домен простой, команда небольшая, а требования стабильны.

Вариант 2 (предлагаемый): Гексагональная архитектура (Ports & Adapters) с элементами DDD-lite

Суть

Ядро (домен + прикладные сценарии) изолировано от инфраструктуры через **порты** (интерфейсы) и **адаптеры** (реализации).

text



Почему именно для этого ТЗ

- **Автоназначение оператора** — естественно ложится в доменный сервис `OperatorAssignmentPolicy` (стратегия: round-robin / least-loaded), тестируется изолированно.

- **Email-уведомления** — порт `NotificationPort` с двумя адаптерами: синхронный (для тестов) и асинхронный (очередь) → снимает конфликт с требованием «≤2 с».
- **Журнал действий** — отдельный порт `AuditPort`, адаптеры: БД, CSV-экспорт. Легко добавить новые форматы.
- **RBAC** — домен-независимый адаптер/декоратор на прикладном слое.
- **Смена БД / добавление FTS** — затрагивает только адаптер репозитория.

Сравнение вариантов

Критерий	Слоистая (Layered)	Гексагональная (Ports & Adapters)
Простота	✅ Высокая. Понятна с первого курса, минимум абстракций, быстрый старт.	⚠️ Средняя. Нужно понимать инверсию зависимостей, порты/адаптеры; больше файлов и интерфейсов.
Тестируемость	⚠️ Средняя. Service тестируется с моками репозитория, но домен «размазан» по сервисам; email/БД часто протекают в бизнес-логику.	✅ Высокая. Домен и use cases тестируются без БД/сети; адаптеры — отдельно (contract-тесты).
Расширяемость	⚠️ Средняя. Новый канал (email-очередь, CSV, внешний API) → правим сервисы; риск «толстых» сервисов.	✅ Высокая. Новый адаптер = новый класс, ядро не трогаем; политики (назначение оператора) — сменные стратегии.
Стоимость внедрения	✅ Низкая. Меньше кода, меньше времени, стандартный стек.	⚠️ Выше. Больше boilerplate (интерфейсы, DI-конфигурация), выше порог входа.
Соответствие учебному проекту	✅ Отлично, если цель — показать базовые паттерны, CRUD, MVC, работу с БД и ролями. Легко уложиться в сроки и защитить.	✅ Хорошо, если цель — продемонстрировать SOLID, DIP, тестируемость, разделение домена и инфраструктуры; но требует больше времени и зрелости команды. Может выглядеть «overengineering» для простого CRUD.

Итоговая рекомендация

- **Если приоритет — сдать в срок и показать рабочий продукт** с понятной структурой → ваш **слоистый вариант** оптимален. Достаточно добавить: `NotificationService` (с интерфейсом), `AuditService`, `AssignmentPolicy` как отдельные классы — это даст 80% выгоды гексагона при 20% затрат.
- **Если приоритет — демонстрация архитектурного мышления, тестируемости и расширяемости** (например, курсовой/диплом по архитектуре ПО) → **гексагональный вариант** выигрывает, особенно из-за требований об уведомлениях, журнале и сменных политиках назначения.

- **Практичный компромисс**: слоистая структура + выделенные порты для «нестабильных» зависимостей (email, аудит, экспорт CSV) + доменный сервис назначения оператора. Это «слоистая с элементами гексагона» — низкая стоимость, хорошая тестируемость ключевых сценариев.

Что стоит уточнить в ТЗ перед выбором архитектуры

1. Алгоритм автоназначения оператора (влияет на доменный сервис/стратегию).
2. Синхронность email-уведомлений и допустимая задержка (влияет на необходимость очереди).
3. Ожидаемый объём данных и нужен ли полнотекстовый поиск (влияет на БД/адаптер).
4. Точный перечень логируемых действий и формат журнала (влияет на `AuditPort`).
5. Безопасность регистрации: пароль в письме — риск, лучше ссылка активации.